

Applied Machine Learning Lab

B.Tech Semester 4

Experiment 10

EMG Gesture Classification using Machine Learning Models

Date: 15th April 2026

SAP ID: 590016154

Name: Vidisha

AIM

To classify hand gestures using EMG signal data and evaluate multiple machine learning models for gesture recognition.

THEORY

Electromyography (EMG) signals measure electrical activity produced by muscles.

Signal Processing: EMG signals are time-series data

Feature Extraction: MAV and RMS

Models: Logistic Regression, KNN, Decision Tree, Random Forest, Extra Trees, SVM

Evaluation: Accuracy and F1-score

Confusion Matrix used for performance analysis

Commands/Code Used

```
# Import libraries
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
```

```
from sklearn.svm import SVC
```

```
from sklearn.metrics import accuracy_score, f1_score
```

```
# -----
```

```
# LOAD DATA (example CSV)
```

```
# -----
```

```
df = pd.read_csv("emg_data.csv") # change path if needed
```

```
# -----
```

```
# FEATURE EXTRACTION (MAV & RMS)
```

```
# -----
```

```

def extract_features(data):

    features = []

    for i in range(len(data)):

        signal = data.iloc[i, :-1].values.astype(float)

        # MAV (Mean Absolute Value)
        mav = np.mean(np.abs(signal))

        # RMS (Root Mean Square)
        rms = np.sqrt(np.mean(signal**2))

        features.append([mav, rms])

    return np.array(features)

X_features = extract_features(df)

# Target
y = df.iloc[:, -1] # last column as label

# -----
# TRAIN TEST SPLIT (70/30)
# -----
X_train, X_test, y_train, y_test = train_test_split(
    X_features, y, test_size=0.3, random_state=42

```

```
)
```

```
# -----
```

```
# SCALING
```

```
# -----
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
# -----
```

```
# MODELS
```

```
# -----
```

```
models = {
```

```
    "Logistic Regression": LogisticRegression(),
```

```
    "KNN": KNeighborsClassifier(),
```

```
    "Decision Tree": DecisionTreeClassifier(),
```

```
    "Random Forest": RandomForestClassifier(),
```

```
    "Extra Trees": ExtraTreesClassifier(),
```

```
    "SVM": SVC()
```

```
}
```

```
results = {}
```

```
# -----
```

```
# TRAIN & EVALUATE
```

```
# -----
```

```

for name, model in models.items():

    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)


    acc = accuracy_score(y_test, y_pred)

    f1 = f1_score(y_test, y_pred, average='weighted')


    results[name] = [acc, f1]


    print(f"\n{name}")

    print("Accuracy:", acc)

    print("F1 Score:", f1)


# -----
# MODEL COMPARISON
# -----

results_df = pd.DataFrame(results, index=["Accuracy", "F1 Score"]).T


results_df.plot(kind="bar", figsize=(10,6))

plt.title("Model Performance Comparison")

plt.xticks(rotation=45)

plt.ylabel("Score")

plt.show()

```

RESULTS

Feature extraction applied on EMG signals.
Multiple models compared using accuracy and F1-score.
Model performance comparison graph generated.

CONCLUSION

EMG data was successfully used for gesture classification. Ensemble models like Random Forest and Extra Trees performed best.